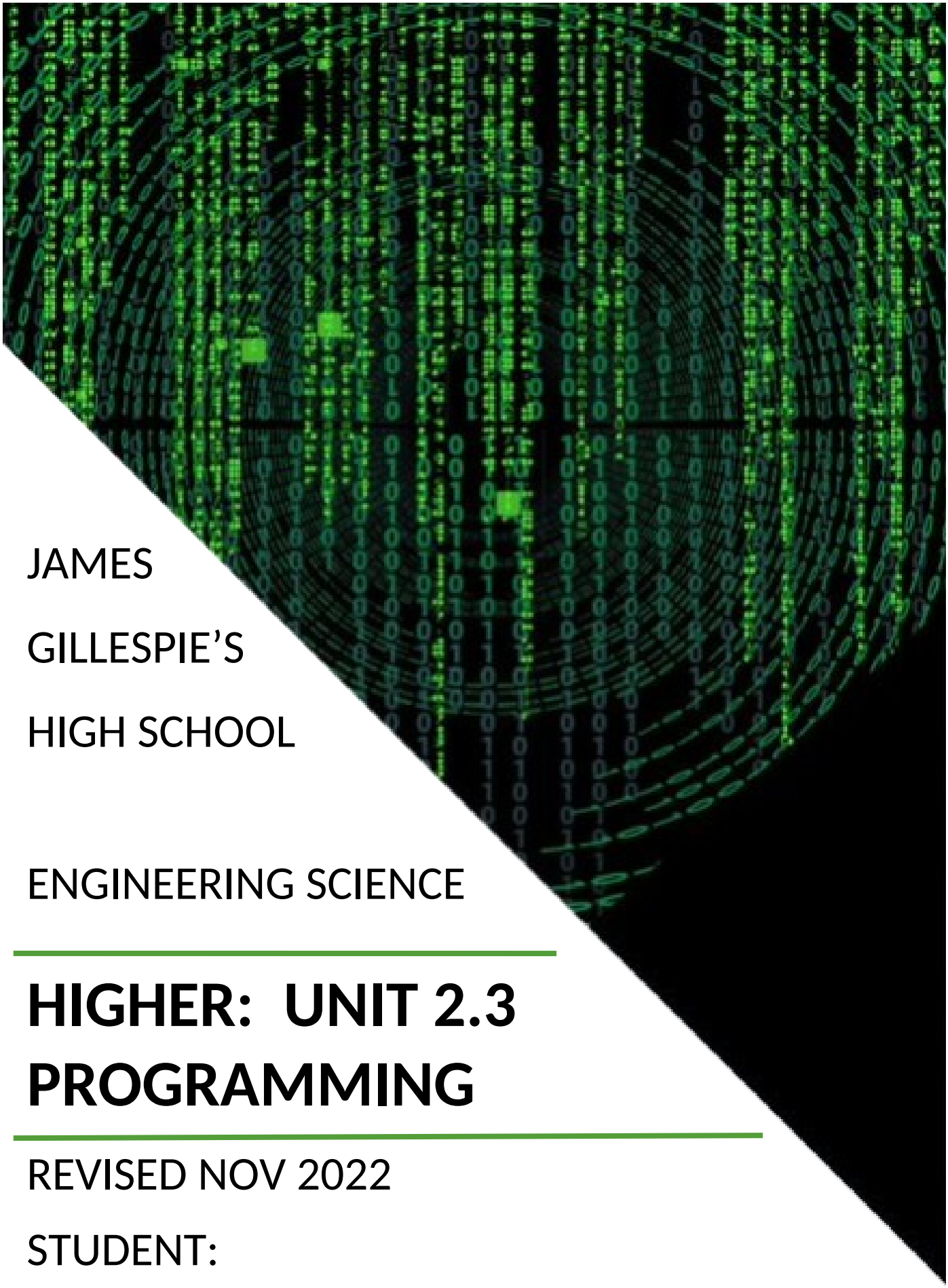


JAMES

GILLESPIE'S

HIGH SCHOOL

ENGINEERING SCIENCE

# HIGHER: UNIT 2.3 PROGRAMMING

REVISED NOV 2022

STUDENT:

YEAR: TEACHER:

# Contents


Flow charts.... 4

Proportional closed loop control.... 5

Microcontrollers.... 5

Microcontroller architecture.... 5

pBASIC coding.... 6

Number systems.... 6

Port addressing and the Data Direction Register (DDR).... 7

Programming revision.... 9

Monitoring multiple inputs.... 11

Flowcharts and PBASIC.... 13

Using symbols.... 14

Further examples.... 14

Loop command.... 15

'If…then' command.... 15

'If… then' command example.... 16

'For…next' command loops.... 17

Sub procedures.... 19

Pulse Width Modulation motor control.... 21

Arithmetic operations.... 23

Analogue input to Digital conversion.... 26

Reference voltage.... 26

Signal conditioning.... 27

Homework questions – to be set by teacher.... 28

Exam style questions.... 29

Additional work and information: Stepper Motor Driver IC.... 33

DC Motor Speed Control.... 33

Digital to Analogue Conversion.... 33

Stepper motor questions.... 34

# Learning Intentions

For programming:

- To control a motor with pulse-width modulation
- To create control routines with up to four inputs and four outputs
- To process analogue inputs
- To use infinite and finite loops and time delays
- To use logic and arithmetic operations to make decisions
- To use high-level programs to monitor inputs and initiate digital outputs
- To use high-level programs to make decisions using arithmetic and logic functions

# Success Criteria

## Skills, knowledge and understanding for course assessment.

To track your progress of learning, consider the above learning intentions and record your progress. This will also help with revision as you will be able to see what areas you need to work on. Your teacher will review this record page after speaking to you about your self-evaluation.

- ☐ I can control a motor with pulse-width modulation.
- ☐ I can create control routines with up to four inputs and four outputs.
- ☐ I can process analogue inputs.
- ☐ I can use infinite and finite loops and time delays.
- ☐ I can use logic and arithmetic operations to make decisions.
- ☐ I can use high-level programs to monitor inputs and initiate digital outputs.
- ☐ I can use high-level programs to make decisions using arithmetic and logic functions.

# Flowcharts

Flowcharts are used to graphically plan a program. Below is a list of recognised flowchart symbols—these are NOT in your data booklet so you must remember these. You will not need some of the symbols until assignments later in this booklet.



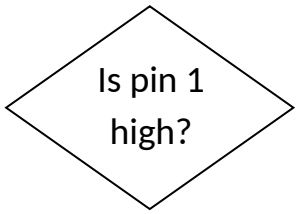

Start/Stop: These symbols are found right at the beginning or right at the end of a flowchart. All flowcharts will contain the start symbol.




Wait: This box indicates a pause or delay in the programme.




Input/Output: This is used to signal when any output is set to be high or low or if a signal is being received from an input.




Decision: This is used to represent whether the program is to make a YES/NO decision.




Gosub: This box is used to direct to flowchart to 'go to sub-routine'. It is also used to begin the sub-routine (start / stop symbol shape) and return to the main programme.



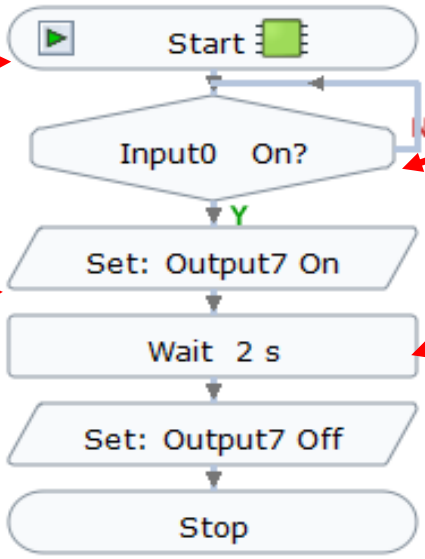
Lines show the direction of flow. Lines going up or left must include an arrow, although it is good practice to include an arrow on every line.



Lines are also used to show loops. Use block arrows heads.

## Flow charts

This shape of box must be used at the start and end of a flow chart.



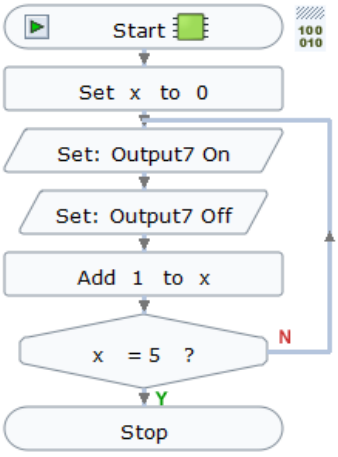

This is a decision box. It asks a question and points where you should go depending on whether the answer is **yes** or **no**.

We use this box to switch things off and on.

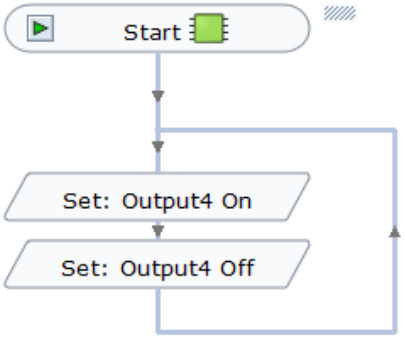
We use this box to pause the flow chart or to do some other internal processes. This time for 2 seconds

We can create loops in two main ways.

1. Finite loops will repeat a section of the flow chart a set number of times. This involves the use of a variable x (x = 5?) to keep track of how many times the loop has been performed.




2. Infinite loops will repeat a section of a flow chart for ever.



## Proportional closed loop control

The main problem with using comparator based control systems is that there is a time lag built into their systems that means the desired level is never actually met, it is really just passed over in both directions, when the set temperature is reached the system will continue to heat up for a short while until it turns off to try and cool down a little. Likewise, when it comes back down to the set value it will take a little while to turn the heater back on. Whilst in heating systems the presence of lag does not present a problem and in fact presents some advantages, in many other applications the inaccuracy of this type of control is not acceptable.

The difference amplifier amplifies the difference between the two signals causing a small error to be read more quickly. This helps prevent the overshoot and time lags in the seen in the comparator-based systems.

# Microcontrollers

A microcontroller is often described as a "computer on a chip". Microcontrollers have memory, processing units and input/output circuitry all built into a single chip. As they are small and inexpensive they can easily be built into other devices to make these products more intelligent and easier to use. Microcontrollers are designed to control specific process or products and are programmed to do complete these tasks. The software however can be changed to allow the same device to be used in a range of different products.

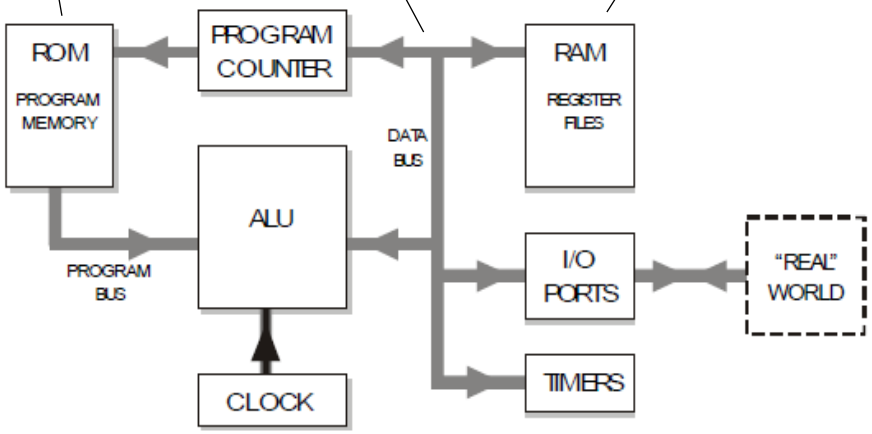
## Microcontroller architecture (Background knowledge. Not requried for exam)

The main features of a microcontroller are shown in the block diagram.

Read Only Memory (ROM) contains operating instructions and is programmed before the m/c is installed in the target system. Information is retained even when the power source is removed.

Buses information is carried between blocks of the m/c by groups of wires called buses.

Random Access Memory (RAM) is used to temporary store information while the program is running, and the information is lost when power is removed.




I/O Ports allow the microcontroller to communicate with the outside world.

The Arithmetic Logic Unit (ALU) is the "brain" of the m/c. It operates by reading instructions from the ROM and then carrying out the mathematical operations for each instruction. The speed at which these operations occur is controlled by the clock circuit.

CLOCK. The clock circuit synchronises all the internal blocks so that the system remains stable.

Timers. Most m/c's have one or more timers built in the most common being the "watchdog" timer, a special timer that resets the m/c if it stops processing for any reason. This ensure the m/c keeps working all the time which is essential in some applications.

## pBASIC coding

The main programming commands that you should be familiar with are used in the program below.

```
SYMBOL red = 7                  ' define pin 7 with the name "red"
INIT:   let dirs = %10000000    ' set up pin 7 as an output, all others as inputs
MAIN:   if pinb.0 = 0 then main ' If input 0 is not on then loop back to the label main
        for b0 = 1 to 5         ' start a for...next loop
          high red              ' switch pin 7 high
          pause 1000            ' wait for 1 second
          low red               ' switch pin 7 low
          pause 1000            ' wait for 1 second
        next b0                 ' end of for...next loop
        end                     ' finish here.
```

## Number systems (Background knowledge. Not requried for exam)

A microcontroller operates by performing many mathematical equations in a short space of time. This is due to information being expressed as a series of electronic signals. These signals are regocnised as being in one of two states, High or Low, It uses the binary number system which consists of only two digits 0 and 1. A high signal is represented as logic 1 and a low signal as logic 0.

The first 16 numbers in the binary system are shown here.

| Dec | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Bin | 0000 | 0001 | 0010 | 0011 | 0100 | 0101 | 0110 | 0111 | 1000 | 1001 | 1010 | 1011 | 1100 | 1101 | 1110 | 1111 |

A single binary digit is referred to as a bit. Different systems carry out calculations using different quantities of bits, so systems are often referred to as 8-bit, 16-bit or 32-bit systems most common is 8-bit. Eight bits grouped together are described as a byte. The decimal value of a byte is calculated by adding the corresponding decimal value of the individual bits. The eight bits in a byte are labelled 0 to 7 from left to right. The bit on the right is the least significant and the bit on the left the most significant bit. Decimal values are shown here.

| Bit number | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| Decimal value | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

The binary number 10010111 converted would be 151

1 0 0 1 0 1 1 1
128 + 0 + 0 + 16 + 0 + 4 + 2 + 1 = 151

### To go from decimal to binary (Background knowledge. Not requried for exam)

Start with a decimal lets say 212 we divide by 2 and each time put down the remainder from the sum then read the result from bottom to top so…

| | | |
|---|---|---|
| 212÷2 | 106 r | 0 |
| 106÷2 | 53 r | 0 |
| 53÷2 | 26 r | 1 |
| 26÷2 | 13 r | 0 |
| 13÷2 | 6 r | 1 |
| 6÷2 | 3 r | 0 |
| 3÷2 | 1 r | 1 |
| 1÷2 | 0 r | 1 |

Read from bottom up
11010100

Notation: When using a different counting system, it is important to distinguish which counting system you are using. For instance, 10 means 2 different things in decimal and binary.

Therefore, the following notations are used in PBASIC programs:

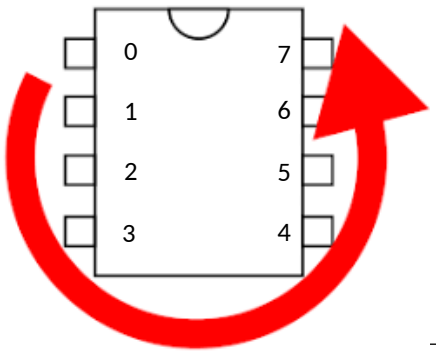
Decimal values are written as 10 (= 10 in decimal).

Binary values are preceded by a % symbol %10 (= 2 in binary).

## Port addressing and the Data Direction Register (DDR)

Microcontrollers communicate with the outside world by input/output pins which are grouped together in “ports”. A Picaxe can have multiple input/output pins, a Picaxe 18M2 has 8 input and 8 output pins.

Each pin can be addressed individually, or all eight simutaneuously. In the PBASIC language the pins are labelled 0 to 7, and the whole port is addressed is allocated the label “pins”. Pins 0-4 are normally the inputs while pins 5-7 are outputs.



For example, to switch pin 3 **"high"** individually the command would be: "**high 3".** To switch pin 3 **"low"** the command would be **"low 3".** Each pin is referred to by a single bit in the port address called "pins".

| Input/ output pin | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| Bit of address "pins" | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Decimal value | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

You read the 'let dirs' pin directory from right to left.
To switch all pins "high" the command would be "**let pins = 255"** to set all pins "low" the command would be **"let pins = 0".** The dirs

To set input pins 0 – 2 as "high", the command would be let pins = **%00000111.**

Note how the use of the binary system can be used on occasion to illustrate which pins are switched high (1) or low (0).

Each pin can be configured to be an output to send digital signals or as an input to receive. The Data Direction Register (DDR) is used to configure the port, and in the PBASIC code language the DDR is allocated the label "let dirs".

- After the code is written, it is <u>important you are able to explain the code</u> by an English translation. It must start with an apostrophe ( ' ). Example: let dirs = %00001111 is explained by 'setting up the directory. <u>You MUST use the apostrophe so the computer code knows to not read it and therefore continues with the coded program only</u>.

If all the bits in the DDR are set high then all the pins will be set as outputs. If all the bits are set low each pin will be an input. For example is the code and then the english translation.

```
let dirs = 255          ' set all pins as outputs.
let dirs = 0            ' set all pins as inputs.
let dirs = %11110000    ' set 0 – 3 as inputs and 4 – 7 as outputs.
```

Each PBASIC program listing should always begin with a "let dirs" statement to correctly setup the DDR. By default all pins are set to inputs when the picaxe is reset.

**1** Write PBASIC commands that will perform the following functions.

a) switch on pins 0 and 1
b) switch on pins 7, 6, 5 and 4.
c) switch on pins 7, 5, 3,and 1.
d) set pin 7 and 6 as outputs and the others as inputs.
e) set pins 6,5 and 3 as outputs and the others as inputs.

**2** (National 5 programming knowledge. Unlikely to be asked at Higher)
Use the internet to research and answer the following:

a) Give three advantages of using microcontrollers in commercial products rather than using discrete electronic devices. (3 marks)

b) Give three disadvantages of using microcontrollers in commercial products rather than using discrete electronic devices. (3 marks)

c) Give the full name and describe the function of the following types of memory; RAM, ROM, EEPROM. (9 marks)

## Programming revision

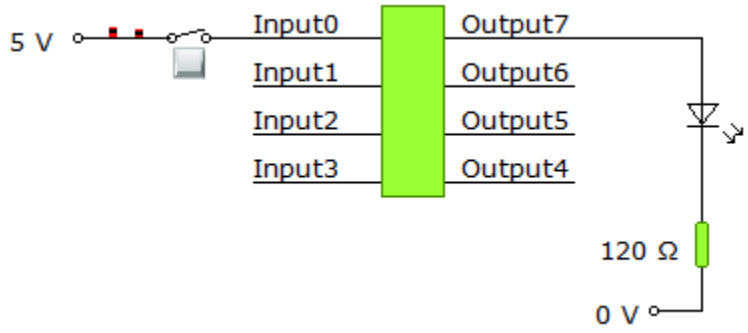
The use of programmable control systems is now common in a huge number of situations. From robotic systems in manufacturing plants, to smart phones, kitchen appliances, games controllers and beyond, they are part of everyday life and will continue to be so for a long time to come.

In the National 5 course you were introduced to the concept of using flowcharts and coding to program microcontrollers to perform set tasks. Here is a reminder of what you should already be familiar with.

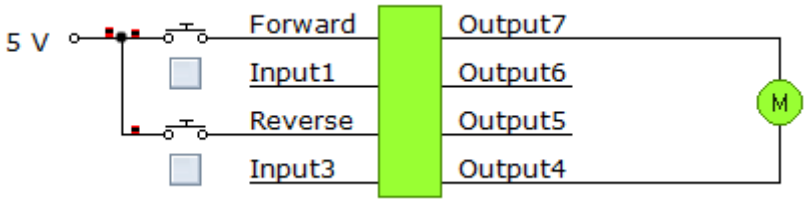
### National 5 Revision question

**3** For the following questions, create a flowchart in your jotter, then produce a working simulation in Yenka. Check and print your solution and paste into your jotter.

a) An LED must flash 10 times after a switch has been pressed.




b) A motor must spin forward when a switch is pressed and reverse when a second switch is pressed.



## National 5 Revision question

**4**

Design a system that will operate the traffic light sequence at a pelican crossing. The system should remain in State 1 until a pedestrian presses a button. In *State 4* the amber light should flash 5 times before returning to *State 1*.

| State 1 | State 2 | State 3 | State 4 |
|---|---|---|---|
|  |  |  |  |

| Input Connection | Pin | Output Connection |
|---|---|---|
| | 7 | Red Light |
| | 6 | Amber Light |
| | 5 | Green Light |
| | 4 | |
| Start Switch | 3 | |
| | 2 | |
| | 1 | |
| | 0 | |

## National 5 Revision questions

**5**

As part of a Christmas decoration, a lighting sequence is to be controlled by a microcontroller. The output connections are shown below.

| Input Connection | Pin | Output Connection |
|---|---|---|
| | 7 | |
| | 6 | Yellow Light |
| | 5 | Purple Light |
| | 4 | |
| | 3 | Blue Light |
| | 2 | Green Light |
| | 1 | |
| | 0 | Red Light |



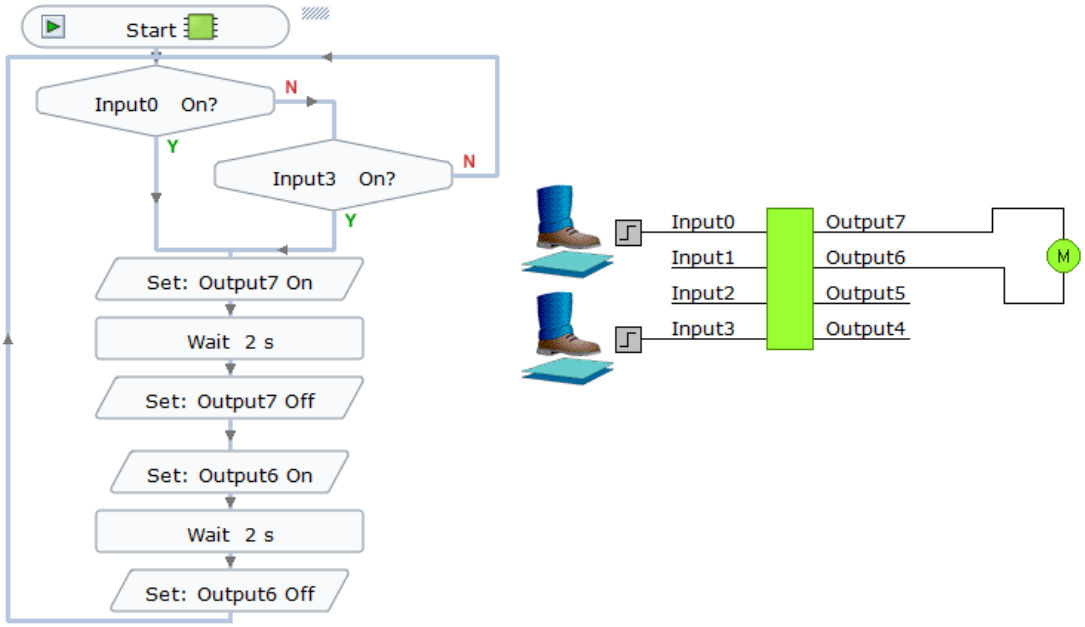
- The red and green lights should come on together and stay on for 5 seconds.
- Then they both go off and the yellow and blue lights should come on together for 8 seconds.
- They then go off and the purple light flashes on and off 6 times (the 'on' and 'off' times being 0.5 seconds each).
- The sequence then repeats itself.

Draw a flowchart for this sequence and model in Yenka.

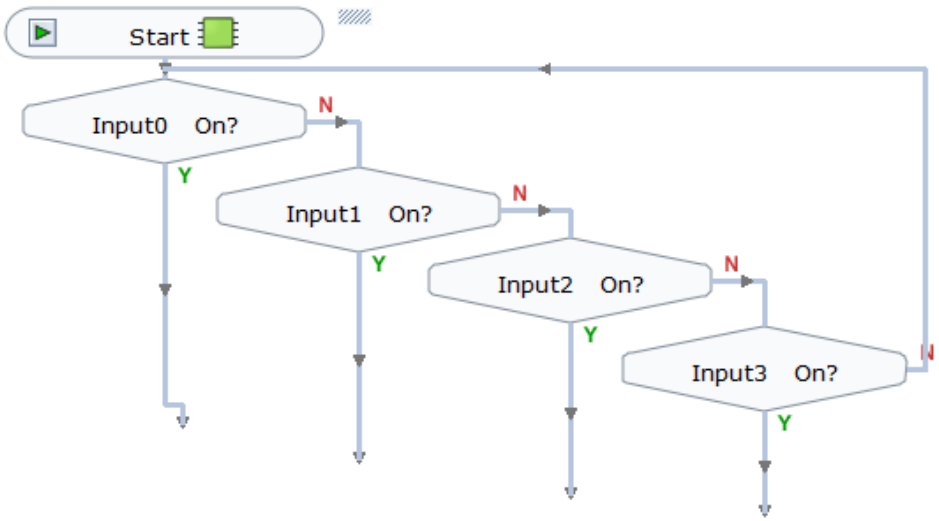
## Monitoring multiple inputs (higher)

There are many situations where systems need to be able to check several inputs at the same time before performing a further function. An obvious case might be an automatic door to a building that would need to check for people entering and leaving a building.

This type of monitoring can be done quite simply in the following way.




The key part here is connecting successive decision boxes in a loop so that each can be checked in turn. This can be expanded for as many inputs as may be required.

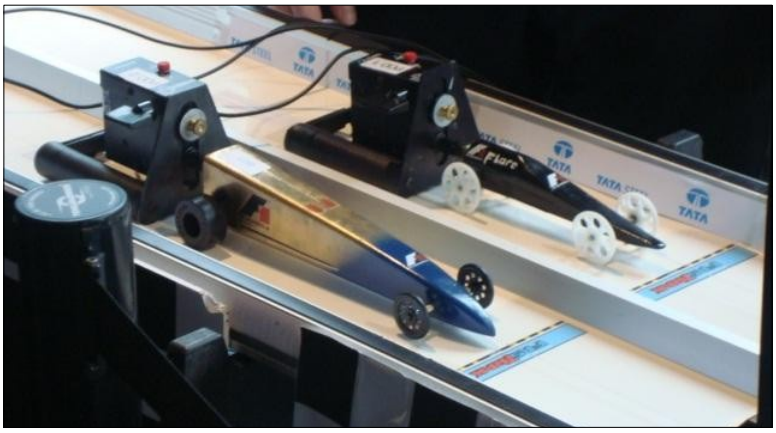


**6** A climate control system for an office must;

- Spin a fan when the temperature rises above a set level (the motor must run off a 12V supply)
- Light a bank of three LEDs when it gets dark.

Create a flowchart of this program in your jotter.

**7** A control system is required for a race track for model dragsters. It is to be a **3-track car** version similar to the two track shown below. Create a flowchart of this program in your jotter.



- The system must be activated by a starter pressing a switch.
- Sensors at the end of each track must activate when that track's dragster goes over them.
- A lamp must light to indicate the track that the winning dragster was on.
- There is a master reset switch which should be pressed at the end of the race to start the racing again. All winning lamps and switched off.

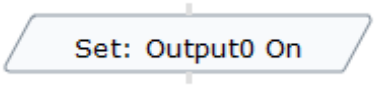
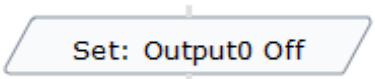
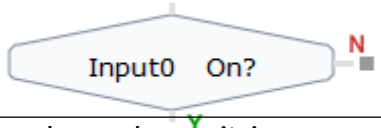
**8** Draw a flowchart of

a) An AND control system.
b) An OR control system.

# Flowcharts and PBASIC

Flow charts are a very useful way of producing simple control programs and helping understand how systems work, but microcontrollers in real-world applications are usually programmed using code.

In Engineering Science, we use a language called PBASIC, but you can also use Arduino. There are several simple commands that you will use that relate closely to the flow chart symbols used so far. The most common are shown in the table below.

| Flow Chart Symbol | PBASIC Command |
|---|---|
|  | high 0 (preferred)<br>*or* high B.0 |
|  | low 0 (preferred)<br>*or* low B.0 |
|  | pause 5000 |
|  | if pin0 = 1 then … |

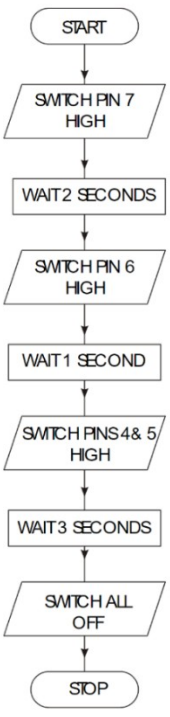
Once a flowchart has been drawn it is necessary to convert it into the stamp controller programming language, which is called PBASIC.

A PBASIC program for the flowchart is shown below. Some flowchart symbols require more than one line of pBASIC code.

A label (for example 'main:' in the program above) can be any word (apart from keywords such as 'high'), but it must begin with a letter. When the label is first defined it must end with a colon (:).

After the programme is the Comments. (an explanation after the apostrophe (') symbol) makes each line of a pBASIC program much easier to understand.

```
main:
high B.7          ' switch pin 7 on
pause 2000        ' wait for 2 seconds
high B.6          ' switch pin 6 on
pause 1000        ' wait for 1 second
high B.5          ' switch pin 5 on
high B.4          ' switch pin 4 on
pause 3000        ' wait for 3 seconds
low B.7           ' switch pin 7 off
low B.6           ' switch pin 6 off
low B.5           ' switch pin 5 off
low B.4           ' switch pin 4 off
end               ' end the program
```



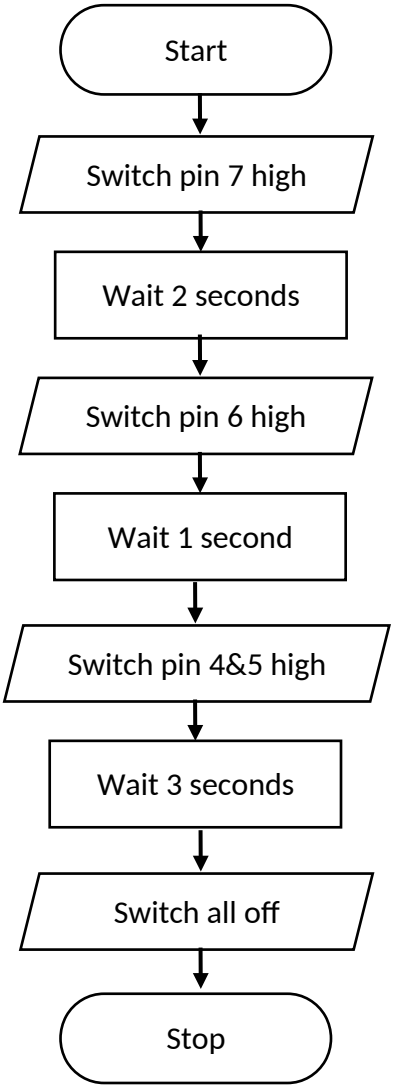
## Using symbols

Sometimes it can be hard to remember which pins are connected to which devices. The 'symbol' command can then be used at the start of a program to rename the inputs and outputs.

```
symbol red = 7                ' rename pin 7 'red'
symbol green = 5              ' rename pin 5 'green'

MAIN:                         ' make a label called 'MAIN'
      high red                ' red LED on
      pause 2000              ' wait 2 seconds
      low red                 ' red LED off
      high green              ' green LED on
      pause 1000              ' wait 1 second
      low green               ' green LED off
      goto MAIN               ' loop "forever"
```

## Further examples

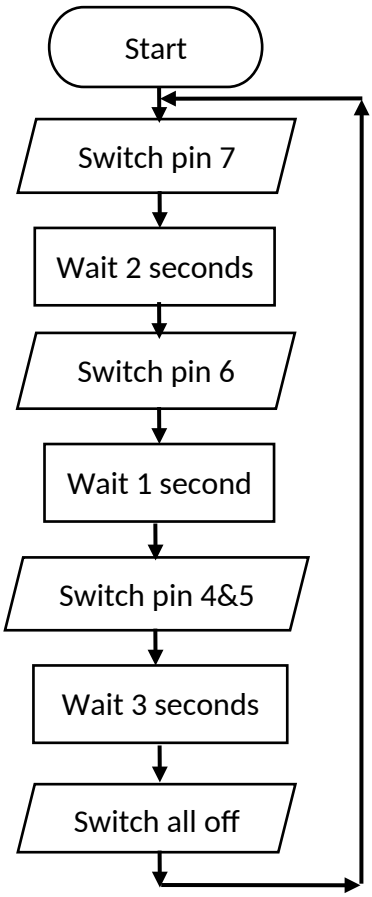



A simple example of a control operation is represented by the flowchart shown. A PBASIC program to achieve this control is shown below.

```
                        let dirs = %11110000     'setting up
                                                 the directory.
                                                 Set pins 0-3
                                                 as inputs and
                                                 pins 4-7 as
                                                 outputs.
MAIN:
      high 7                  'set pin 7 on
      pause 2000              'wait for 2 seconds
      high 6                  'set pin 6 on
      pause 1000              'wait 1 second
      let pins = %11110000    'set pins 4-7 on
      pause 3000              'wait 3 seconds
      let pins = 0            'switch all pins off
      end                     'end the program
```

## Loop command

Sometimes it's necessary to make programs that loop "forever" as shown opposite. In this case we need to add labels to the program and to use the "goto" command to 'jump' to the line marked by the label.




```
                                    let dirs = %11110000     'setting up
                                                             the directory.
                                                             'set pins 0-3
                                                             as inputs and
                                                             pins 4-7 as
                                                             outputs
MAIN:
      high 7                  'set pin 7 on
      pause 2000              'wait for 2 seconds
      high 6                  'set pin 6 on
      pause 1000              'wait 1 second
      let pins = %11110000    'set pins 4-7 on
      pause 3000              'wait 3 seconds
      let pins = 0            'switch all pins off
      goto MAIN               'loop "forever"
```

## 'If...then' command

The following portion of a program makes pin 7 flash every time a switch on input pin 0 is pushed.

```
MAIN:                               ' make a label called 'MAIN'
      if pin0 = 1 then FLASH        ' jump to the FLASH label if input pin0 is on
      goto MAIN                     ' else loop back around

FLASH:                              ' make a label called 'FLASH'
      high 7                        ' switch output 7 on
      pause 500                     ' wait 0.5 second
      low 7                         ' switch output 7 off
      goto MAIN                     ' jump back to start
```

In this program the first three lines make up a continuous loop. If the input is off, the program just loops around time and time again. If the switch (pin0) is then pushed, the program english translation 'jumps' to

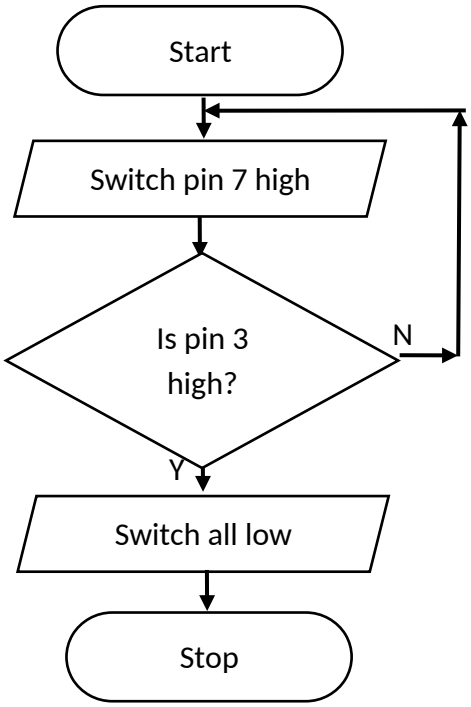
the label called ‘FLASH’. The program then flashes output 7 on for half a second before returning to the main loop.

Note carefully the spelling in the ‘if … then’ line – ‘pin0’ = 1 is all one word (without a space). Note also that only the label is placed after the command ‘then’ – no other words apart from a label are allowed (FLASH).

Example: if pin0 = 1 then FLASH

## ‘If… then’ command example

The ‘if..then’ programming structure allows the computer to make a decision based on information received from an input pin. Look at the following flowchart and program.




```
let dirs = %10000000          ‘setting up the directory

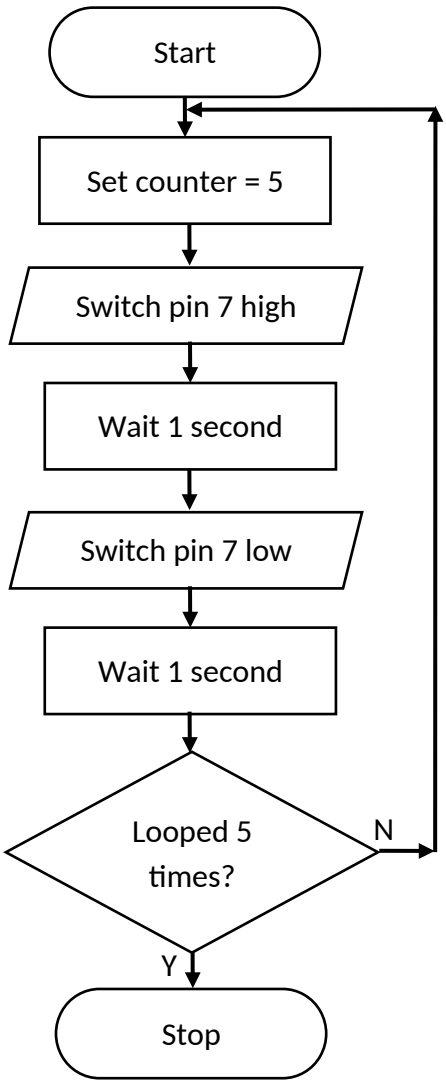
MAIN:                         ‘make a label called MAIN
      let pins = %10000000    ‘switch pin 7 on
      if pin 3 = 1 then SKIP  ‘jump to ‘SKIP’ if input 3 is on
      goto main               ‘loop back to the label called MAIN
SKIP:
      let pins = 0            ‘switch all pins off
end                           ‘end the program
```

This program switches on pin 7 and then waits for an input connected to pin 3 to go high. When the input switch is actuated the output pin is switched low. Notice how the program loops when there is a negative response to “is pin 3 high” and will do continuously until pin 3 is high. It will then proceed to the next command.

## ‘For…next’ command loops

It is often useful to repeat the same part of a program a number of times, for instance when flashing a light. In these cases, a ‘for … next’ loop can be used to repeat a section of code a number of times.

The number of times the program runs is set by a variable. A variable is a number that is stored in the RAM memory of the stamp controller. There are 14 memory locations that byte variables can be stored in. These location are labelled b0 to b13 but can also be renamed to give more appropriate names by use of the “symbol” command.




```
let dirs = %10000000     ‘setting up the directory. Set
                         pin 7 as output on
symbol counter = b0      ‘define the variable “counter”
symbol red = 7           ‘define pin 7 with name “red”

MAIN:
            for counter = 1 to 5     ‘start a for…next
      loop
            high red                 ‘switch pin 7 on
            pause 1000               ‘wait 1 second
                  low red                  ‘switch pin 7
            off
            pause 1000               ‘wait 1 second
            next counter             ‘end of for…next loop
end                            ‘end program
```

Note the extra white space (extra dots) has been used to clearly illustrate all the commands between the “for” and “next” commands.

The symbol command is also used to label pins and variables to make them easier to use.

## 9

A set of temporary traffic lights is required for a system of road works.

| | |
|---|---|
| red | 10 s |
| red and amber | 2 s |
| green | 10 s |
| amber | 2 s |



Write a pBASIC program, using 'high' and 'low' commands, to achieve this operation. Use the following pin configuration: red (7), amber (6) and green (5).

## 10

Go back over Q3-7 and write pBASIC programs for the flowcharts you created.

## 11

A toy shop has a train set in the window. The train set has an electric train, a set of red/green signals and a set of moving track points that allow the train to move around two different loops of track.

The toy-shop owner wants to develop a program that will carry out the following sequence.

1. Switch the green signal on
2. Switch the train on for 3 seconds
3. Stop the train
4. Switch the green signal off and the red signal on
5. Switch the points on
6. Switch the train on for 3 seconds
7. Stop the train
8. Switch the points off
9. Switch the red signal off and the green signal on
10. Jump back to step 1

| Input connection | Pin | Output connection |
|---|---|---|
| | 7 | train motor |
| | 6 | points |
| | 5 | green signal |
| | 4 | red signal |
| | 3 | |
| | 2 | |
| | 1 | |
| | 0 | |

Draw a flowchart for the control sequence and then use the flowchart to write a pBASIC program.

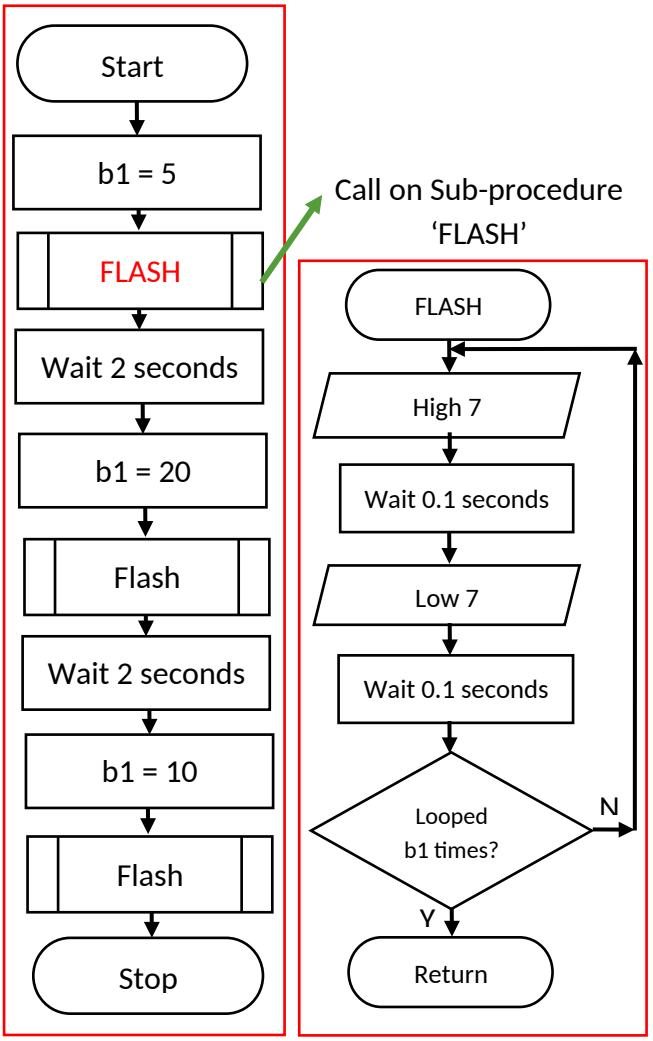
## Sub procedures

It is often useful to be able to re-use sections of code within a program. A sub-program is a small section of code that can be "called" from a different part of theprogram. After the sub-program is finished the program flows back to the original section of the program. The flowchart symbol is shown below and indicated the program should 'jump' to the sub-proceedure. To "call" a sub-procedure, the **gosub** (go sub) command is used. The last line of the sub-procedure must be **return** which means return to the original program.

```
Initial: let dirs = %10000000 'setup the DDR

MAIN:
      let dirs. = %10000000   'switch pin 7 high
      let b1 = 5              'give b1 value 5
      gosub FLASH             'call sub-procedure
      pause 2000              'wait 2 seconds
      let b1 = 20             'give b1 value 20
      gosub FLASH             'call sub-procedure
      pause 2000              'wait 2 seconds
      let b1 = 10             'give b1 value 10
      gosub FLASH             'call sub-procedure
end                           'end MAIN program

'sub-procedure starts here
FLASH:
      for b2 = 1 to b1  'setup a loop using b1
      high 7            'output pin on
      pause 100         'wait 100ms
      low 7             'output pin off
      pause 100         'wait 100ms
      next b2           'next loop
return                  'return from sub-procedure
```




In this example the sub-procedure is used to actually flash the LED on and off. The number of flashes is set by variable b1, which is set before each sub-procedure is called. By changing the value of b1 the number of flashes can be changed.

Note the end command between the main program and the sub-procedure. This is needed to make sure it stops the controller accidentally "running into" the sub-procedure after the main program has been completed.

## 12

As part of a shop display, a lighting sequence is to be controlled by a microcontroller. The output connections are shown below.

| Input Connection | Pin | Output Connection |
|---|---|---|
| | 7 | Red Light |
| | 6 | Yellow Light |
| | 5 | Green Light |
| | 4 | Orange Light |
| | 3 | |
| switch 2 | 2 | |
| switch 1 | 1 | |
| | 0 | |

If switch 1 is pressed, each light should flash on and off 5 times in sequence i.e. orange then green then yellow then red (each 'on' and 'off' time should be 0.5 seconds). the system then resets for the next switch push.
If switch 2 is pressed all lights should flash simultaneously 3 times. There should then be a 2 second pause, before all lights should flash simultaneously 6 times (each 'on' and 'off' time should be 0.5 seconds). The system then resets for the next switch push.

Draw a flowchart and write a PBASIC program for this sequence (use a sub-procedure for the flash routines).

## 13

A sequence of events for the operation of the washing machine is given in the table below:

| | |
|---|---|
| Step 1 | Wait until the door switch is closed. |
| Step 2 | Wait until the start switch is pushed. |
| Step 3 | Rotate the drum clockwise for 10 seconds. |
| Step 4 | Rotate the drum anti-clockwise for 10 seconds |
| | *Repeat steps 3 and 4 10 times* |
| Step 5 | Rotate the drum clockwise for 5 seconds. |
| Step 6 | Rotate the drum anti-clockwise for 5 seconds |
| | *Repeat steps 5 and 6 5 times* |

Draw a flowchart for the control sequence described above.

Write, and test, a PBASIC program for the control sequence described above.

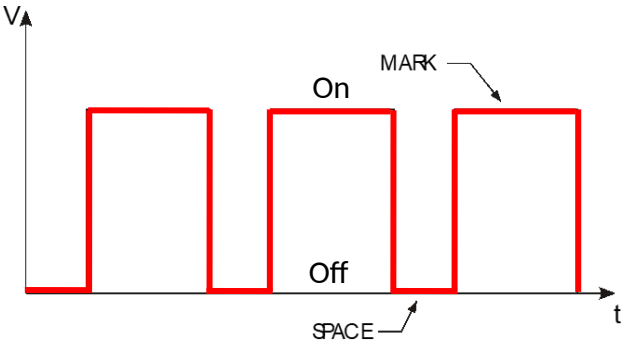
# Pulse Width Modulation motor control

There are two ways to control the speed of a d.c. motor.

1. Vary the voltage supply ie: if 3v is applied to a motor it will rotate slower than if 5v were applied the problem with this is that the lower the voltage supply the lower the torque reducing the power of the motor. Note, this is not simple to do with a digital system because the outputs of a microcontroller are either 5V or 0V, nothing in between. Therefore, this solution is often unsatisfactory for controlling DC motors due to the undesirable loss of motor torque.

2. Second way to control speed is Pulse Width Modulation (PWM), a method of switching power off and on rapidly (pulsing). By varying the on and off ratio of the pulses the speed of the motor can be varied. As the full voltage is applied to the motor during the 'on' pulses the torque of the motor remains high.

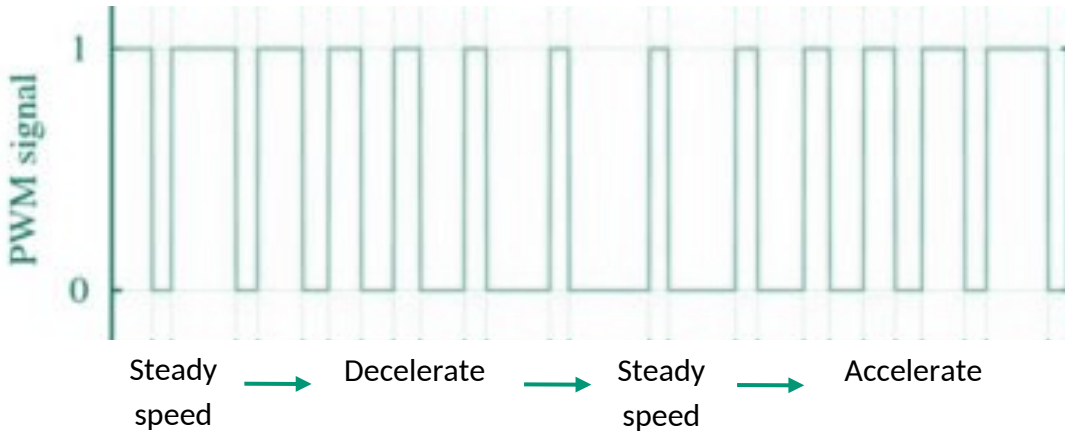
The graph on the right shows how the technique is applied. The 'on' time for the motor is called the mark, the 'off' time is called the space. When the voltage is applied to the motor it accelerates to top speed. However, before the top speed is reached the motor is switched off, thus slowing it down. By increasing the frequency of the pulses this acceleration/deceleration becomes negligible, and the motor rotates constantly at a slower speed.




The speed of the motor can be increased by increaing the mark shortening the space. Doing the opposite will slow down the motor.

The PWM technique does have certain limitations. It cannot be used with mechanical relays, as the rapid switching would damage the mechanical contacts. The frequency of the pulses must also be carefully selected, as if the frequency is too slow the motor will stall.

In some devices, such as electric drills, it is desirable for the motor to start rotating slowly and then build up speed, rather than rapidly 'accelerating' up to full speed. This is called 'soft starting' the motor, and the use of PWM is often appropriate in these situations. The motor is started at a low speed and then gradually accelerated by varying the mark to space ratio over a period of time.



Below is an example of a PBASIC program for PWM motor control.

```
symbol mark = b1
symbol space = b2
symbol motor = 7

initial:
     let dirs = %11110000          ' pin 4-7 outputs
     let mark = 10                 ' set mark to 10ms
     let space = 20                ' set space to 20ms
MAIN:
     high motor                    ' output motor on
     pause mark                    ' pause for 10ms (mark time)
     low motor                     ' output motor off
     pause space                   ' pause for 20ms (space time)
     goto MAIN                     ' loop
```

**14** Sketch a graph to show how pulse width modulation could be used to gradually increase the speed of a motor from rest. Your graph should include at least 6 pulses.

**15** Using Yenka, download the following programme and answer the 5 questions from the website in your jotter.

https://www.yenka.com/activities/PWM_Speed_Control_of_d.c._Motor_-_Activity/

(or go to Yenka.com and search pwm, select "PWM Speed Control of d.c. Motor task") Once you have completed this task, if you feel confident create your own PWM program and try it out. Try out different speeds (by experiment) by altering the values of 'mark' and 'space'.

**16** An automatic sliding door must meet the following specification.

- The door is operated by a motor with limit switches feeding back when it is fully open and fully closed.
- The door should open whenever a person is sensed.
- The door should remain open for five seconds.
- The door should close at half speed using PWM.
- The sequence repeats.

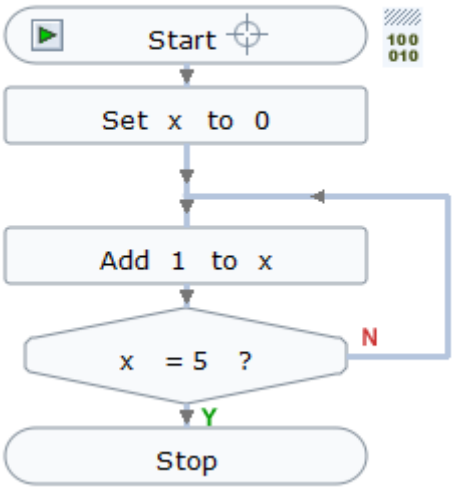
a) Draw a flowchart for the control of the door motor with reference to the specification

b) With reference to the pinout table below, write a PBASIC program to control the operation of the motor.

| Input | Pin | Output |
|---|---|---|
| | 7 | Motor forward (open door) |
| | 6 | Motor back (close door) |
| | 5 | |
| | 4 | |
| | 3 | |
| Person sensor (1 = person sensed) | 2 | |
| Limit switch A (1 = door closed) | 1 | |
| Limit switch B (1 = door open) | 0 | |

# Arithmetic operations

Finite loops can use arithmetic operations to test if a variable has reached a desired value. In the flowchart below this is done using the commands *add 1 to x* and *x = 5?*.



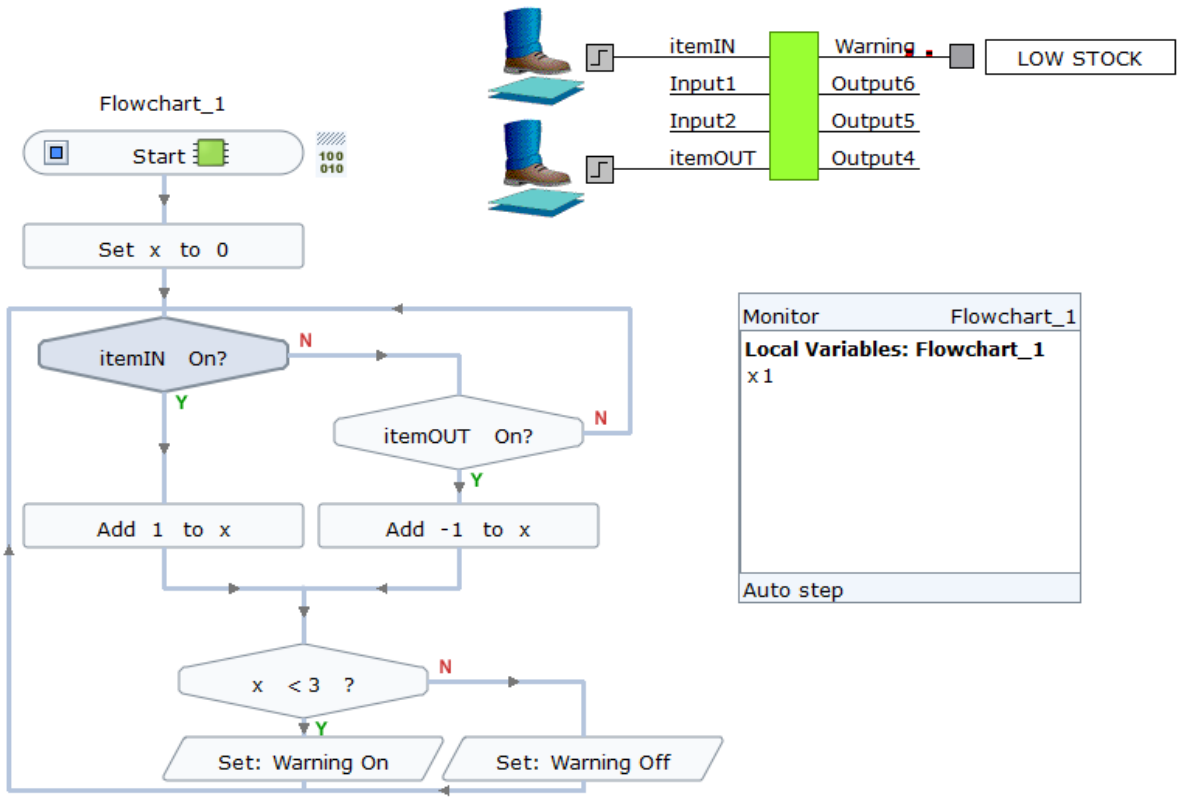

We can use a number of other operations to give us more control. The table below shows a number of commands and how they could be programmed on Yenka or in PBASIC.

| Command | Yenka | PBASIC |
|---|---|---|
| Decrease the value of x | Add -1 to x | let x = x-1 |
| Divide the value of x by 3 (this will return a whole number) | Set x to x/3 | let x = x/3 |
| Multiply the value of x by 3 | Set x to x*3 | let x = x*3 |
| Check if x is more than 5 | x > 5 ? N Y | if x > 5 then label |
| Set the value of x to half of y | Set x to y/2 | let x = y/2 |

## 17

The system below shows how a warehouse could simply keep track of the number of items in stock. As one new item is brought in a sensor is activated and the value of x (a variable that indicates the number of items) is incremented. A second sensor is activated when an item is removed and sent to a customer.

The final part of the flow chart shows how a warning could be issued when stock levels are low. Using Yenka model the system shown below and use the monitor box to see how x reacts to the input from the sensors. Write a PBASIC program to perform the same function as the system shown.




## 18

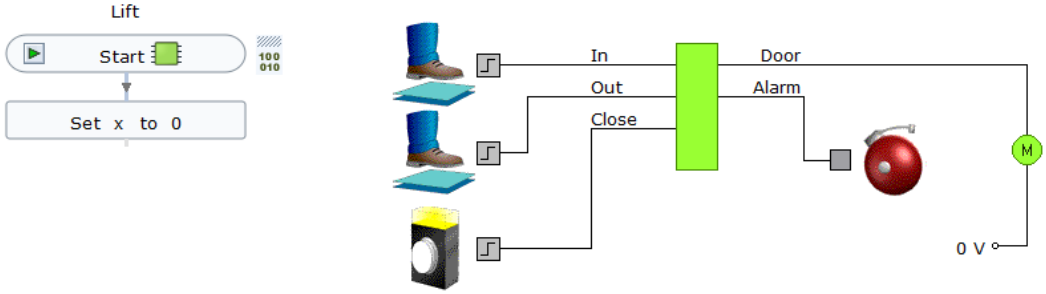
Develop a system in Yenka and a PBASIC program to keep track of the number of cars in a car park. When there are 10 a barrier should close.

## 19

A system is required to control a lift in a building. It must;

- Count people coming in and out of the lift.
- If there are 10 or less in the lift when a close button is pressed, then a motor closes the door
- If there are more than 10 people when the 'close' button is pressed an alarm must switch on.
- The alarm must stop when there are 10 people or less in the lift. (Someone needs to get out)

Complete a Yenka simulation and PBASIC program to perform the required specification.

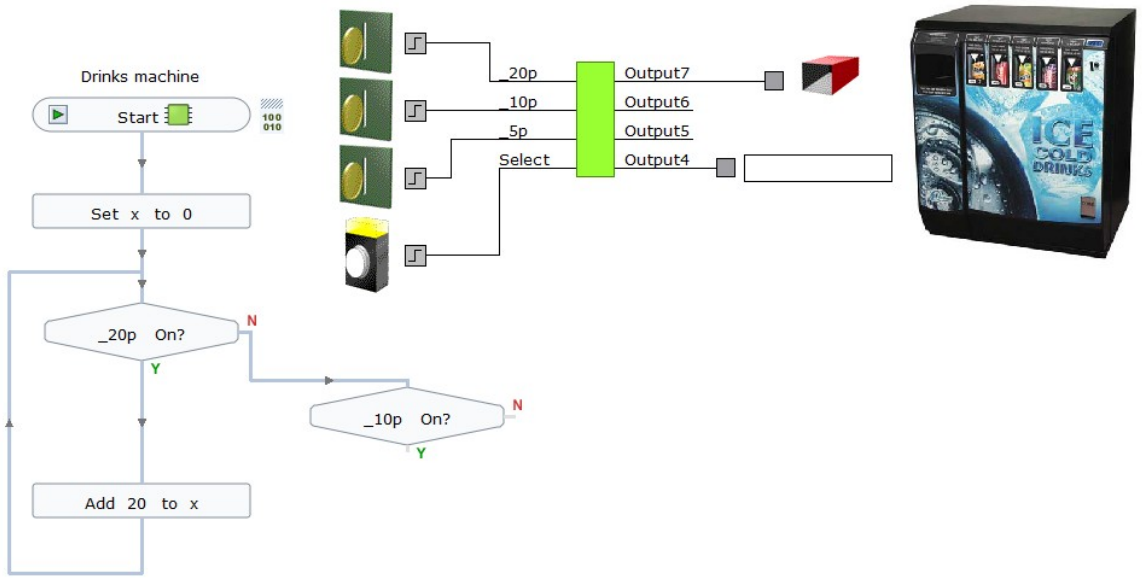


# 2

A drinks machine can take in the following coins: 5p, 10p and 20p. There will be a separate slot for each type of coin. Create a system that will meet the following criteria.

- When 35p is put in the machine and a button is pressed a can must be given out.
- The machine must give out change in 5p pieces. (Use a lamp to indicate each coin being ejected).

Complete the Yenka simulation shown below and a PBASIC program to perform the required specification.

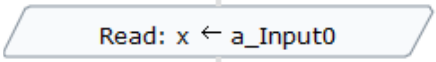


# Analogue input to Digital conversion

So far you have made use of digital inputs but in the real world there are numerous situations when an analogue input is received and must be processed, such as heat or light. A games console will need to take in very detailed information from a motion sensor to work out where a user is and what movements they are making. This would not be possible if the sensor only indicated if there was movement or not.

The microcontroller can only process digital (high/low) signals. An analogue–to-digital convertor (ADC) is a device that can convert analogue quantities into digital signals. An 8-bit ADC can generate a digital signal in the range 0-255 (one byte), so it converts the input value to a binary number in that range (0 - %11111111). ACD's are available as separate integrated circuits or may even be built into the microcontroller to give it "on board" ADC capabilities.

They will convert the input analogue value to a binary number in the range of 0-255 (0 - %11111111).

Analogue signals can be read by a control system in Yenka using 


This will take a reading from an input pin that has been set as an analogue input and stores it in variable x.

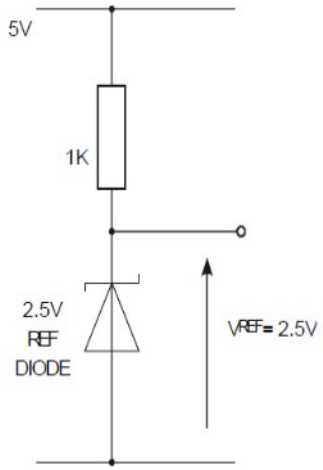
In PBASIC the same function can be performed with the command,

readADC 0,x

This works with microcontrollers that have inbuilt ADCs on 1 or more of their inputs. For the instruction above the 0 indicates that pin0 is an analogue input with an analogue-to-digital converter and that the analogue input value will be stored in variable x.

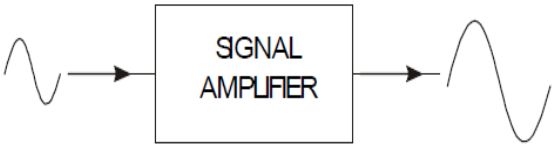
## Reference voltage

ADC's are designed to process analogue signals within a certain range. The maximum voltage signal that can be processed by an ADC is called a reference voltage. A common reference voltage for an 8-bit ADC is 2.55V, so the ADC can measure signals in the range 0 to 2.55V. Therefore, the 8-bit ADC device will generate an 8-bit value directly equivalent to the voltage signal applied.



## Signal conditioning

Analogue sensors, such as the thermistor, may not directly produce a suitable signal for use with the ADC. It is often necessary to use a signal amplifier circuit so that the input signal can be "conditioned" to the input range of the ADC (0 to 2.55V). An operational amplifier, configured as a voltage amplifier, is commonly used for this task. You learned about these in the analogue electronics unit.




An example of signal conditioning could be when a thermistor is used to monitor the temperature of a factory furnace. Experimentation has shown that the thermistor produces a linear output signal reading of 0V at 0° Celsius and 1.53V at 200° Celsius. The maximum operating temperature of the furnace is 1000° Celsius. The maximum voltage that may be fed into the ADC is 2.55V.

To solve this problem, first of all calculate the maximum input signal, which is at 1000° Celsius.

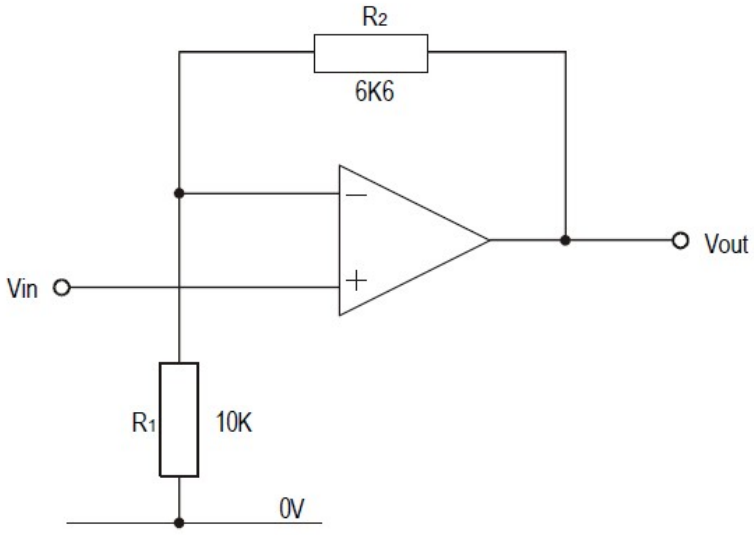
Signal at 200° = 1.53V ➡ 1000° = 5 x 200° so maximum signal is 1.53 x 5 = 7.53V

Next calculate the gain required required from the op-amp circuit.

Input signal = 7.65V
Required output signal = 2.55V

Gain = output/input = 2.55/7.65 = 0.33

So we can construct a circuit diagram of a voltage amplifier with gain of 0.33




**21** Design a system, and produce a PBASIC program, that will perform the following function for a projector.

- When a start switch is pressed a lamp comes on.
- If the temperature rises above a desired level a fan is switched on.

# Homework questions – to be set by teacher

## Homework question 1

Due date:_____.

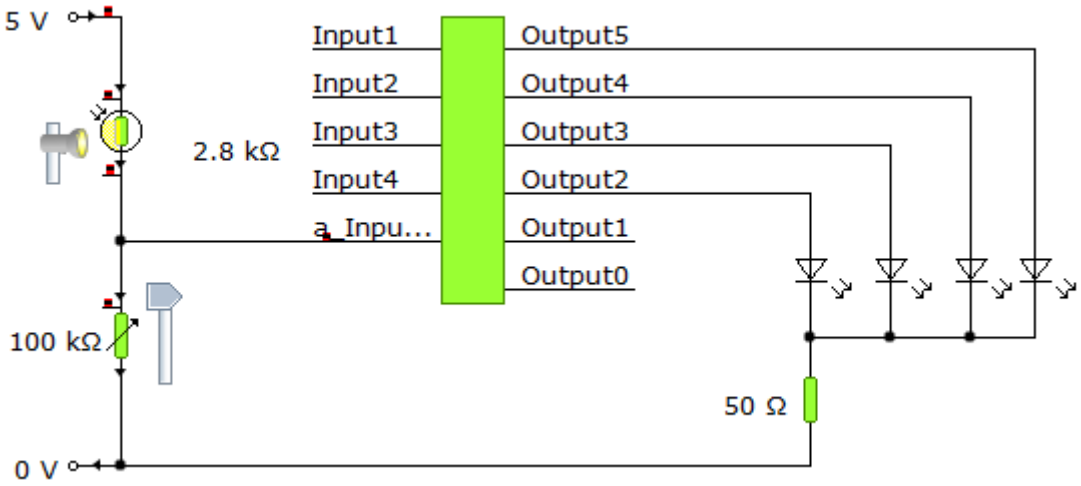
Write a PBASIC program to perform the following function;

- increase the speed of a motor when one switch is pressed
- decrease when a second switch it pressed
- stop the motor when a third is pressed

## Homework question 2

Due date:_____.

As the outside light level decreases 4 LEDs come on in succession to maintain a consistent light level in a room. Complete a Yenka simulation and a PBASIC program.




## Homework question 3

Due date:_____.

A wind turbine uses a potentiometer to sense the direction of the wind. It uses a second potentiometer to monitor which direction the turbine is facing.

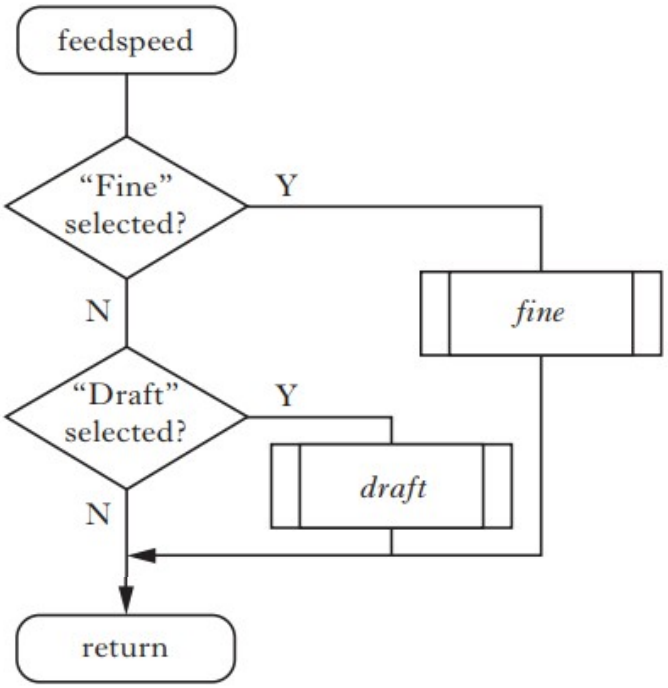
a) Design a control system in Yenka, and write a PBASIC program, that will make a motor turn forward or backward to move the turbine to turn into the wind.
b) Adjust the speed of the motor so that it slows down as it gets closer to its desired position.

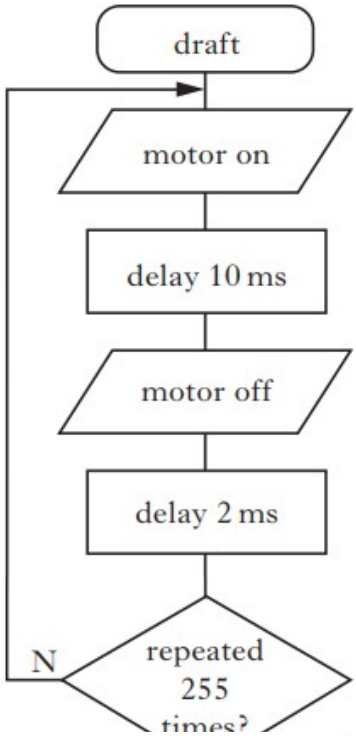
# Exam style questions

Q1

The speed at which the paper feeds through a printer is selected by a PBASIC sub-procedure *feedspeed*. The sub-procedures *fine* and *draft* control the speed of the paper-feed motor using pulse-width modulation (PWM), for two different feed rates.

A flowchart for *feedspeed* is shown in Figure Q1(*a*), and for *draft* in Figure Q1(*b*). The relevant microcontroller connections are shown in Figure Q1(*c*).



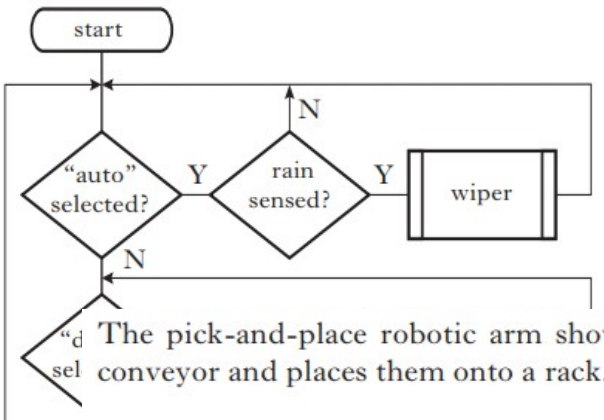




The windscreen-wipers of a car have three modes of operation controlled by a switch which sends signals to a microcontroller. The switch is mounted on a stalk beside the steering wheel, as shown in Figure Q5(*a*).




Figure Q5(*a*)

The control sequence is represented by the flowchart shown in Figure Q5(*b*).




7

5

Q2

Q3

The pick-and-place robotic arm shown in Figure Q7(*a*) lifts components from a conveyor and places them onto a rack. The arm is controlled by a microcontroller.

29

Q4
From Specimen Q10(b)

A car manufacturer uses a pneumatic system during the production of car doors.

After the doors have been assembled, an automated system is used to ensure quality checks are made.

Part of the system operates on the following sequence:

- A motor must switch on to move doors along a conveyor;
- A light sensor detects when the door has reached the inspection point;
- A pneumatic piston outstrokes for 1 second and diverts every fourth door for inspection;
- If an operator presses the sampling switch when a door is in position it will also be diverted using the piston;
- When a door is diverted a checking light flashes on and off 6 times over 3 seconds;
- This system continually repeats.

The relevant microcontroller connections are shown in the table below.

| Inputs | Pins | Outputs |
|---|---|---|
| | 7 | Pneumatic piston |
| | 6 | Conveyor motor |
| | 5 | Checking light |
| | 4 | |
| | 3 | |
| | 2 | |
| Sampling switch | 1 | |
| Light sensor | 0 | |

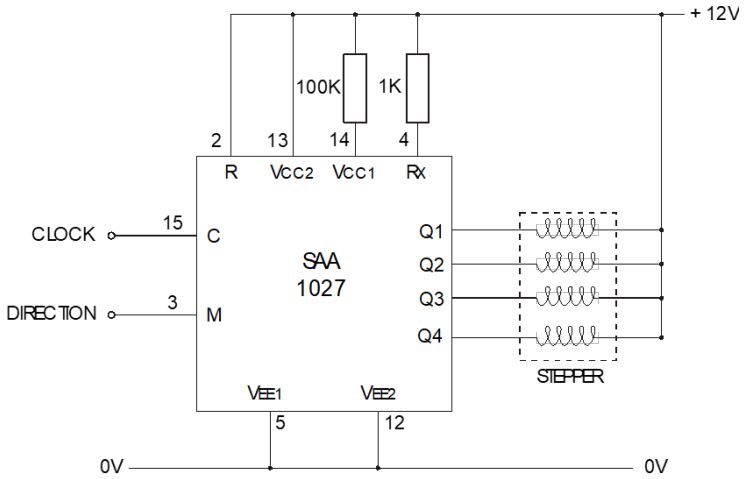
Light sensor (High signal indicates a door.)

Draw a flowchart to show the control of the quality checks.

# Additional work and information: Stepper Motor Driver IC

Stepper motors are not required knowledge for higher, however if you would like to discover more about motor control, research stepper motors and their control compared to PWM (work done on pg 19) and attempt the questions below.

The use of four output pins to drive a stepper motor can be an inefficient way to use the microcontroller input/output pins. A dedicated integrated circuit, called the SAA1027 stepper motor driver, has been designed to overcome this problem by building the logic step generator, and the transistor switches, into one package.




The stepper motor driver IC just requires two signals - direction and step. The 'direction' signal sets the direction of rotation, when set high the motor turns one way, when set low the motor turns the other way. The 'step' signal is pulsed (switched high then low again) to move the stepper motor one step. Therefore to move the stepper motor ten steps the 'step' pin would be pulsed ten times.

## DC Motor Speed Control

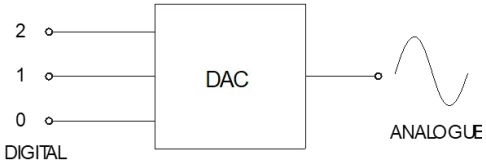
The speed of a dc motor varies directly with the voltage applied across it. Microcontrollers do not normally have an analogue output, and so they cannot be used to vary the voltage to a motor in this manner. However, there are two main methods by which a microcontroller can control the speed of a DC motor:

- Digital to Analogue Conversion (DAC)
- Pulse Width Modulation

## Digital to Analogue Conversion

The simplest way to control the speed of a DC motor is to vary the voltage applied to the motor coils - the higher the voltage the faster the motor will spin (within the motor operating limits).

However the digital output from a microcontroller is at a fixed voltage, and the microcontroller cannot supply enough current to drive the motor. Therefore an interfacing circuit is required to boost the supply current for the motor and to provide different voltage levels.



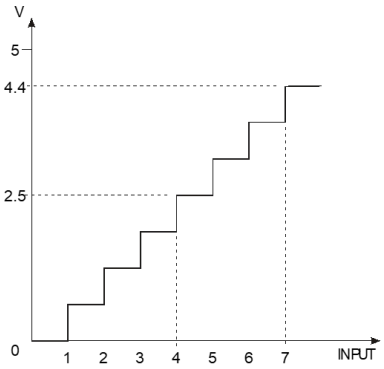

| binary input | analogue output (V) |
|---|---|
| 000 | 0.00 |
| 001 | 0.62 |
| 010 | 1.25 |
| 011 | 1.87 |
| 100 | 2.50 |
| 101 | 3.12 |
| 110 | 3.75 |
| 111 | 4.37 |

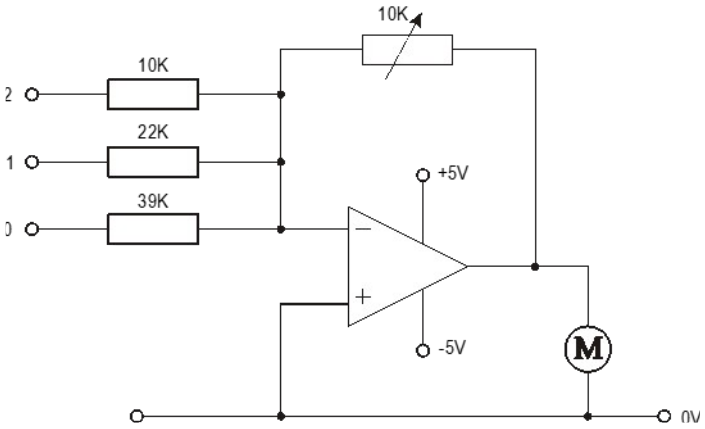
A **Digital to Analogue Converter (DAC)** is an integrated circuit that decodes binary information and generates an analogue voltage proportional to the binary information provided.

A three bit DAC, with an output range of 0-5V, may produce a voltage output according to the table below:

Each increase in the binary input produces a step increase of 0.625V (5V ÷ 8 steps) in the analogue output. In most practical applications the DAC cannot supply the full output voltage (e.g. 5V) due to the technical operating limitations of the device.

When plotted as a graph this transfer function appears as a 'staircase' shape. An increase in the number of input bits will increase the resolution of the DAC, therefore producing 'smaller steps'. The simplest form of DAC is made from a summing amplifier, as shown in the diagram below.




In this example the output voltage is equal to

Vo = -5 (Pin 2 x (10K/10K) + Pin 1 x (10K/22K) + Pin 0 x (10K/39K))
= -5 (Pin 2 + 0.5 (Pin 1) + 0.25 (Pin0))
where the pin value is 0 or 1 (presuming a high logic signal of 5V).

## Stepper motor questions

**A** A system is required to make a stepper motor make one eighth of a revolution then pause for 1 second before commencing the next turn. This should be repeated eight times after a start switch and a safety switch have been pressed. The motor moves in 7.5° per step and uses a step rate of 0.125 seconds (i.e. maximum of eight pulses per second). Draw a flow chart for this sequence (8 marks) and complete a PBASIC program using the following code (12 marks).

```
symbol direction = pin7
symbol pulse = pin6
symbol counter = b0
symbol counter2 = b1

let dirs = %11000000
let pins = 0
let direction = 0

main:
```

**B** A system is required to make a motor turn for 10 seconds then pause for 1 second before turning in reverse for 10seconds. This should be repeated four times. A switch must be pressed to start the sequence. Draw a flow chart (12 marks) for this sequence and complete a PBASIC program on the right (8 Marks).

```
symbol counter = b0
```

```
let dirs = %11000000
main:
```